GENERATIVE AI & AGENTIC AI · LESSON 10 OF 11

Customising Models: Prompting vs RAG vs Fine-Tuning, LoRA, Distillation and Open-Weight Models

Decide how to adapt a model to your task. The customisation ladder (prompting, few-shot, RAG, tools, fine-tuning, training), what fine-tuning is good and bad at, preparing training data, parameter-efficient fine-tuning with LoRA and QLoRA (with the maths), distillation from a large to a small model, running open-weight models locally with Ollama and at scale with vLLM, and the cost, privacy and quality trade-offs of API vs self-hosting.

GenAI Launchpad

✦ 2026: LoRA/QLoRA, Ollama, vLLM, open-weight Llama, Mistral, Qwen, DeepSeek, Gemma

Advanced ⏱ 100 min

By the end of this lesson you can

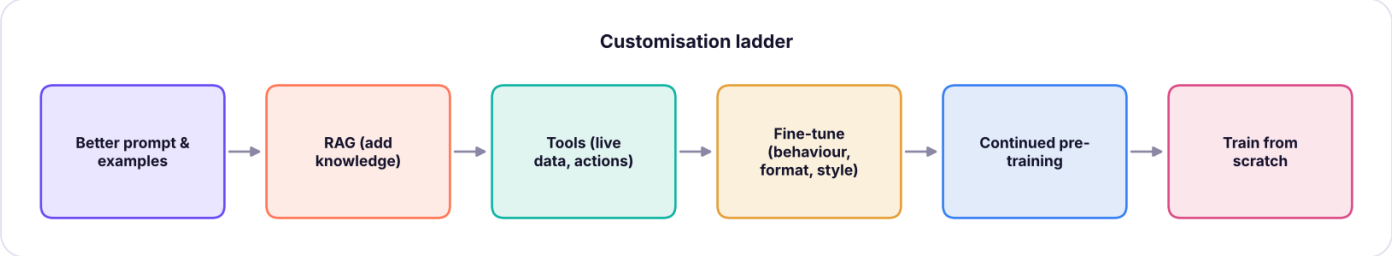
- Choose between prompting, RAG, tools and fine-tuning for a requirement
- Prepare a fine-tuning dataset in chat format and split it for evaluation
- Explain LoRA/QLoRA and compute their parameter savings
- Describe distillation and when a small specialised model wins
- Run open-weight models locally and compare API and self-hosting costs

Datasets used

`customer_reviews.csv`

The customisation ladder

Climb only as far as you need: each step costs more to build and maintain.



From cheapest to most expensive ways to adapt an LLM.

Need	Best first approach
Model lacks your **knowledge** (policies, products, documents)	RAG
Needs **live data** or must **act**	Tools / MCP
Must follow a **format or style** consistently	Prompt + examples + structured outputs; fine-tune if still unreliable
A **narrow task at huge volume** must be cheap and fast	Fine-tune or distil a small model
Specialised **domain language** (legal Hindi, medical codes)	Better model choice, RAG; then fine-tune
Model must run **on-premises / offline**	Open-weight model (optionally fine-tuned)

WATCH OUT **Fine-tuning is not a knowledge database**

Fine-tuning teaches patterns (tone, format, task behaviour). It is unreliable for injecting facts, which change and need citations: use RAG for knowledge. Most teams that "need fine-tuning" first need better prompts, examples and evaluation.

Preparing fine-tuning data

Fine-tuning datasets are examples of the exact behaviour you want, usually in **chat format** (JSONL: one conversation per line).

Python

```python
import json, numpy as np, pandas as pd
rv = pd.read_csv("customer_reviews.csv")
SYSTEM = "Classify the review's sentiment as positive, negative or neutral. Reply with one word."
def to_example(text, label):
    return {"messages": [{"role": "system", "content": SYSTEM}, {"role": "user", "content": text}, {"role": "assistant", "content": label}]}
data = rv.sample(frac=1, random_state=0)
train, val, test = np.split(data, [int(.8 * len(data)), int(.9 * len(data))])
with open("train.jsonl", "w", encoding="utf-8") as f:
    for r in train.itertuples(): f.write(json.dumps(to_example(r.review_text, r.sentiment), ensure_ascii=False) + "\n")
print(len(train), "train |", len(val), "validation |", len(test), "test")
print(open("train.jsonl", encoding="utf-8").readline()[:220])
print("label balance:", train.sentiment.value_counts(normalize=True).round(2).to_dict())
import os; os.remove("train.jsonl")
```

OUTPUT

```
2400 train | 300 validation | 300 test
{"messages": [{"role": "system", "content": "Classify the review's sentiment as positive, negative or neutral. Reply with one word."}, {"role": "user", "content": "Bahut badiya. Sound is really clear and loud, setup took
label balance: {'positive': 0.53, 'negative': 0.32, 'neutral': 0.15}
```

Data quality rules: hundreds to a few thousand **high-quality, diverse, correctly labelled** examples beat tens of thousands of noisy ones; remove duplicates and PII; cover edge cases; keep a test set the model never sees; and compare against the best prompt-only baseline.

Full fine-tuning vs LoRA

Full fine-tuning updates every weight: billions of parameters, lots of GPU memory. **LoRA** (low-rank adaptation) freezes the original weights and learns a small update for selected matrices as the product of two thin matrices:

$$W' = W + \Delta W = W + BA, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k},\ r \ll \min(d, k)$$

Python

```python
def lora_params(d, k, r): return r * (d + k)
d = k = 4096                                                    # one attention projection
in a ~7-8B model
for r in (4, 8, 16, 64):
    print(f"rank {r:2d}: {lora_params(d, k, r):,} trainable vs {d * k:,} full ({100 *
lora_params(d, k, r) / (d * k):.2f}%)")
layers, mats = 32, 4                                            # 32 transformer layers × q,
k, v, o projections
total_full = 8e9; total_lora = layers * mats * lora_params(4096, 4096, 16)
print(f"whole model: LoRA r=16 trains {total_lora / 1e6:.1f}M parameters ≈ {100 *
total_lora / total_full:.2f}% of 8B")
print(f"memory for weights in 16-bit: {total_full * 2 / 1e9:.0f} GB | 4-bit (QLoRA):
{total_full * 0.5 / 1e9:.0f} GB")
```

OUTPUT

```
rank  4: 32,768 trainable vs 16,777,216 full (0.20%)
rank  8: 65,536 trainable vs 16,777,216 full (0.39%)
rank 16: 131,072 trainable vs 16,777,216 full (0.78%)
rank 64: 524,288 trainable vs 16,777,216 full (3.12%)
whole model: LoRA r=16 trains 16.8M parameters ≈ 0.21% of 8B
memory for weights in 16-bit: 16 GB | 4-bit (QLoRA): 4 GB
```

Python

```python
rng = np.random.default_rng(0)
true_update = rng.normal(size=(64, 4)) @ rng.normal(size=(4, 64))           # a genuinely
low-rank change
U, S, Vt = np.linalg.svd(true_update + rng.normal(0, .01, (64, 64)))
for r in (1, 2, 4, 8):
    approx = (U[:, :r] * S[:r]) @ Vt[:r]
    print(f"rank-{r} approximation keeps {100 * (1 - np.linalg.norm(true_update - approx) /
np.linalg.norm(true_update)):.1f}% of the update with {r * 128} numbers instead of 4,096")
```

OUTPUT

```
rank-1 approximation keeps 27.0% of the update with 128 numbers instead of 4,096
rank-2 approximation keeps 43.5% of the update with 256 numbers instead of 4,096
rank-4 approximation keeps 99.8% of the update with 512 numbers instead of 4,096
rank-8 approximation keeps 99.7% of the update with 1024 numbers instead of 4,096
```

Fine-tuning updates tend to be low-rank, so a small rank captures most of the change. **QLoRA** loads the frozen base model in 4-bit precision, so a 7–8B model can be fine-tuned on a single consumer or Colab GPU.

Python

```python
# Typical open-model LoRA fine-tune (Hugging Face) - run on a GPU (e.g. Colab)
from datasets import load_dataset
from peft import LoraConfig
from trl import SFTTrainer, SFTConfig
ds = load_dataset("json", data_files={"train": "train.jsonl", "validation": "val.jsonl"})
trainer = SFTTrainer(
    model="Qwen/Qwen2.5-1.5B-Instruct",                  # small open model
    train_dataset=ds["train"], eval_dataset=ds["validation"],
    peft_config=LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05, target_modules="all-
linear"),
    args=SFTConfig(output_dir="sentiment-lora", num_train_epochs=2,
per_device_train_batch_size=8, learning_rate=2e-4),
)
trainer.train(); trainer.save_model()                    # saves only the small adapter
```

Tools: Hugging Face TRL + PEFT, Unsloth (fast, memory-efficient), Axolotl, and managed fine-tuning on cloud platforms (Amazon Bedrock, Google Vertex AI, Azure AI Foundry, OpenAI) for supported models.

Distillation: big teacher, small student

Use a large model to **label or generate** high-quality training data, then fine-tune a small, cheap model on it. For a narrow, high-volume task (classifying millions of tickets), the student can approach the teacher's accuracy at a fraction of the cost and latency.

Python

```python
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score
teacher_acc = 0.93                                       # suppose a large LLM labels
reviews with ~93% accuracy
noisy = train.copy(); flip = rng.random(len(noisy)) > teacher_acc
noisy.loc[flip, "sentiment"] = rng.choice(["positive", "negative", "neutral"], flip.sum())
for name, labels in [("human labels", train.sentiment), ("teacher (LLM) labels",
noisy.sentiment)]:
    student = make_pipeline(TfidfVectorizer(ngram_range=(1, 2), min_df=2),
LogisticRegression(max_iter=2000, C=5)).fit(train.review_text, labels)
    print(f"student trained on {name:22s} → test accuracy {accuracy_score(test.sentiment,
student.predict(test.review_text)):.3f}")
```

OUTPUT

```
student trained on human labels           → test accuracy 0.943
student trained on teacher (LLM) labels   → test accuracy 0.923
```

Here a tiny classical "student" trained on teacher labels nearly matches one trained on human labels. The same idea, with a small LLM as the student, powers many cost-efficient production systems. (Check each provider's terms: some prohibit using outputs to train competing models.)

Open-weight models

Model families (2026)	Notes
Llama (Meta)	Widely supported, many sizes
Mistral / Mixtral	Efficient, Apache-licensed options
Qwen (Alibaba)	Strong multilingual and coding models
DeepSeek	Strong reasoning, MoE architectures
Gemma (Google), Phi (Microsoft)	Small models for laptops and edge devices
Indic models (Sarvam, Krutrim, AI4Bharat)	Indian-language focus

Run locally with **Ollama** (`ollama run llama3.2` , then an OpenAI-compatible API at `localhost:11434`), LM Studio or llama.cpp; serve at scale with **vLLM**, TGI or SGLang. Always check the **licence** (commercial use, user limits) and evaluate on your task.

API vs self-hosting

Python

```python
def api_cost(req, tin, tout, pin, pout): return req * (tin * pin + tout * pout) / 1e6
monthly_requests = 3_000_000; tin, tout = 400, 60
api_small = api_cost(monthly_requests, tin, tout, 0.10, 0.50)        # small hosted model (Claude Haiku 5.5 list price, USD/M tokens)
api_large = api_cost(monthly_requests, tin, tout, 4.00, 20.00)       # frontier model
gpu_month = 1.2 * 24 * 30                                             # one cloud GPU at ~$1.2/hour, always on
engineering = 2500                                                    # share of an engineer's time to run it (USD/month, illustrative)
print(f"API small model: ${api_small:,.0f}/month | API frontier model: ${api_large:,.0f}/month | self-host 1 GPU: ${gpu_month + engineering:,.0f}/month (incl. ops)")
```

OUTPUT

```
API small model: $210/month | API frontier model: $8,400/month | self-host 1 GPU:
$3,364/month (incl. ops)
```

Factor	Hosted API	Self-hosted open model
Quality	Frontier models	Good to very good; best for narrow tasks
Cost	Pay per token; zero at zero use	Fixed GPU + engineering; cheap only at high, steady volume
Privacy	Contracts, zero-retention options, regional hosting	Data never leaves your infrastructure
Speed to ship	Minutes	Weeks
Maintenance	Provider updates	You patch, scale, monitor and evaluate

Small hosted models are often cheaper than self-hosting until volumes are very large; self-hosting wins on strict data residency, offline/edge needs, or very high steady volume.

Summary

KEY TAKEAWAYS

- Climb the ladder: prompt → examples → RAG → tools → fine-tuning; use RAG for knowledge and fine-tuning for behaviour.
- Fine-tuning needs clean, diverse chat-format examples, a held-out test set and a prompt-only baseline.
- LoRA trains small low-rank updates (often < 1% of parameters); QLoRA adds 4-bit base weights for single-GPU training.
- Distillation trains a small model on a large model's outputs for cheap, fast, narrow tasks.
- Open-weight models (Llama, Mistral, Qwen, DeepSeek, Gemma) run via Ollama/vLLM; compare API and self-hosting on quality, total cost, privacy and effort.

Quick quiz

6 questions · tap an answer to check it

1. Your assistant must answer from 5,000 changing policy pages. Best approach?

- **A** RAG
- **B** Fine-tuning
- **C** Training from scratch
- **D** A higher temperature

Answer: A. Knowledge belongs in retrieval.

2. What does LoRA train?

- **A** Small low-rank matrices added to frozen original weights
- **B** All model weights
- **C** Only the tokenizer
- **D** The embedding database

Answer: A. Few trainable parameters.

3. What does QLoRA add?

- **A** A 4-bit quantised frozen base model, cutting memory
- **B** More training data
- **C** A new tokenizer
- **D** RAG

Answer: A. Fine-tune big models on one GPU.

4. What is distillation?

- **A** Training a small student model on a large teacher model's outputs
- **B** Removing stop words
- **C** Compressing PDFs
- **D** Prompt chaining

Answer: A. Cheap, fast specialised models.

5. When is self-hosting an open model most attractive?

- **A** Strict data residency/offline needs or very high, steady volume
- **B** A prototype with 100 users
- **C** When you need the best possible quality immediately
- **D** When nobody can manage GPUs

Answer: A. Include engineering costs.

6. What is the first thing to try before fine-tuning for a formatting problem?

- **A** Clear instructions, examples and structured outputs
- **B** Training from scratch
- **C** Buying GPUs
- **D** Removing the system prompt

Answer: A. Cheapest rung first.

EXERCISE **Practice**

(1) Write 20 high-quality chat-format examples for a ticket-routing task. (2) Compute LoRA trainable parameters for a 70B model with 80 layers, hidden size 8,192, rank 8 on q/v projections. (3) Install Ollama and compare a small open model with an API model on 20 reviews. (4) Calculate the monthly volume at which one GPU becomes cheaper than the small API model.

Show solution (worked answer)

Python

```
print(80 * 2 * lora_params(8192, 8192, 8) / 1e6, "M parameters")          # ≈ 21M
per_req = (400 * 0.10 + 60 * 0.50) / 1e6
print(f"break-even ≈ {(gpu_month + engineering) / per_req:,.0f} requests/month")
```

INTERVIEW CORNER **Questions you may be asked**

Q: RAG or fine-tuning? RAG for knowledge that changes or needs citations; fine-tuning for consistent behaviour, format, tone or narrow tasks at scale. They combine well: a fine-tuned model can still use RAG.

Q: Explain LoRA in one minute. Instead of updating a huge weight matrix, LoRA learns two small matrices whose product approximates the needed change; the original weights stay frozen, so training is cheap and adapters are small and swappable.

Q: How would you decide between an API model and a self-hosted model? Evaluate quality on the task, total monthly cost including engineering, latency, data-privacy and residency requirements, licence terms and the team's ability to operate GPUs.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.